Birla Institute of Technology & Science, Pilani Hyderabad Campus

CS F372: Operating Systems

Part 1: A Technical Guide to Diffusing Fork Bombs (v6.14)

0. Background:

A fork bomb is a type of Denial of Service (DoS) attack where a process continually replicates itself until the system runs out of resources (CPU time and process slots), causing it to freeze or crash.
Key Features:

1. Replication: A single process executes a command to "fork" (copy) itself.
2. Exponential Growth: Both the parent and the new child process then fork themselves again.
3. Exhaustion: This creates an explosion of processes that completely overwhelms the Operating System.

In Unix-like systems, a fork bomb can be written in just a few characters: **:(){ :|:& };:**

- ***:()*** Defines a function named ***:***
- ***{ :|:& }*** The function calls itself and pipes the output into another background copy of itself.
- ***;:*** Closes the definition and starts the first process.

1. Baseline:

Before modifying the kernel to diffuse fork bombs, it is important to realize that modern versions of the kernel are quite resistant to these fork bombs out of the box.

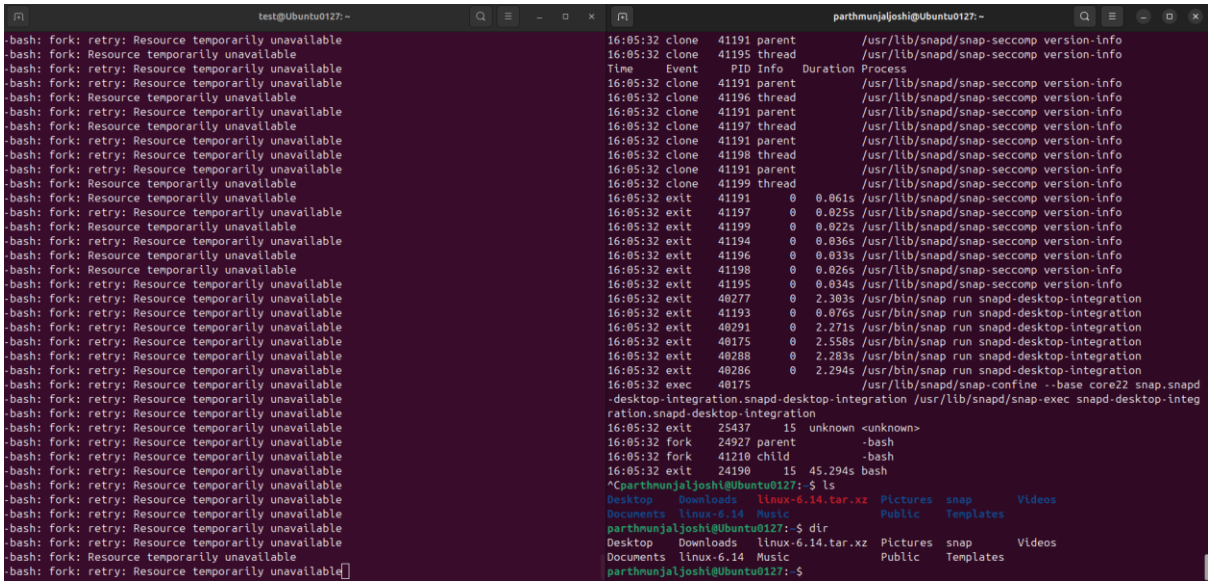
To test this fork bomb we will first create a non-root user. The following syntax allows us to create a user and to switch to that user:

```
parthmunjaljoshi@Ubuntu0127:~$ sudo adduser test
[sudo] password for parthmunjaljoshi:
info: Adding user `test' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `test' (1001) ...
info: Adding new user `test' (1001) with group `test (1001)' ...
info: Creating home directory `/home/test' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for test
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] Y
info: Adding new user `test' to supplemental / extra groups `users' ...
info: Adding user `test' to group `users' ...
```

```
parthmunjaljoshi@Ubuntu0127:~$ su - test
Password:
test@Ubuntu0127:~$
```

Now we shall have two terminals open, one with the new user “test” and one with the root user. We will keep these side-by-side and run ***sudo forkstat*** on the root terminal and then the fork bomb i.e. **:(){ :|:& };:** on the test user terminal.

This is what I observed:
Initially both terminals freeze this is because the CPU is 100% occupied trying to create new processes and context switching between thousands of tiny tasks. Then a kernel safety feature called *Process Limits* are triggered and the kernel begins rejecting further fork() requests. This leads to the following error being continuously visible on the test user terminal:
-bash: fork: retry: Resource temporarily unavailable
After this the admin shell becomes responsive again as test is not consuming as many process ids and system resources because of the Process Limits. However, the test terminal keeps spamming the error message and the only way to stop it is to kill that terminal.



After our modification to the kernel, the fork bomb will be automatically detected and killed giving us back control of our test terminal. This will be the important distinction from the baseline above.

You can check the process limits that saved the system from complete unresponsiveness using the command:

```
parthmunjaljoshi@Ubuntu0127:~$ ulimit -a
real-time non-blocking time  (microseconds, -R) unlimited
core file size              (blocks, -c) 0
data seg size               (kbytes, -d) unlimited
scheduling priority                 (-e) 0
file size                   (blocks, -f) unlimited
pending signals                     (-i) 15333
max locked memory           (kbytes, -l) 501224
max memory size             (kbytes, -m) unlimited
open files                          (-n) 1024
pipe size                (512 bytes, -p) 8
POSIX message queues         (bytes, -q) 819200
real-time priority                  (-r) 0
stack size                  (kbytes, -s) 8192
cpu time                   (seconds, -t) unlimited
max user processes                  (-u) 15333
virtual memory              (kbytes, -v) unlimited
file locks                          (-x) unlimited
```

You can verify that both root and test user will have the same ulimits by default.

2. Getting the Kernel Source Tree

We again need the kernel source tree. You can see your existing kernel version using **uname -r**
It is recommended you download a source tree of a similar kernel family 6.X as uname -r.
The steps for the same are reproduced here from the assignment 1 document.
To install linux-6.14 source code we can use:
wget https://www.kernel.org/pub/linux/kernel/v6.x/linux-6.14.tar.xz

```
parthmunjaljoshi@Ubuntu0127:~$ uname -r
6.14.0-37-generic
parthmunjaljoshi@Ubuntu0127:~$ wget https://www.kernel.org/pub/linux/kernel/v6.x/linux-6.14.tar.xz
--2026-01-27 06:54:48--  https://www.kernel.org/pub/linux/kernel/v6.x/linux-6.14.tar.xz
Resolving www.kernel.org (www.kernel.org)... 151.101.129.55, 151.101.65.55, 151.101.1.55, ...
Connecting to www.kernel.org (www.kernel.org)|151.101.129.55|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 149408504 (142M) [application/x-xz]
Saving to: ‘linux-6.14.tar.xz’

linux-6.14.tar.xz             100%[=============================================>] 142.49M  4.41MB/s    in 22s

2026-01-27 06:55:19 (6.39 MB/s) - ‘linux-6.14.tar.xz’ saved [149408504/149408504]
```

To extract the source tree from the tar archive you can use:

```
~$ tar -xvf linux-6.14.tar.xz
```

Consider the following directory to be our base directory now onwards. We will stay in this directory for all our commands that follow below till the modified kernel is installed.

```
~/linux-6.14
```

3. TIF Flags:

To mark a process as a fork bomb, consistently and efficiently, we define a new TIF flag. TIF (Thread Info Flags) are low-level bitmask flags stored in a thread's hardware-specific structure (typically struct thread_info) that notify the Linux kernel that a specific task needs immediate attention.

To define our TIF for working with fork bomb diffusion: **gedit *./arch/x86/include/asm/thread_info.h***
(This is architecture specific your path might be different on arm machines)

Open *thread_info.h ~/linux-6.14/arch/x86/include/asm

```
96 #define TIF_UPROBE              12      /* breakpointed or singlestepping */
97 #define TIF_PATCH_PENDING       13      /* pending live patching update */
98 #define TIF_NEED_FPU_LOAD       14      /* load FPU on return to userspace */
99 #define TIF_NOCPUID             15      /* CPUID is not accessible in userland */
100 #define TIF_NOTSC               16      /* TSC is not accessible in userland */
101 #define TIF_NOTIFY_SIGNAL       17      /* signal notifications exist */
102 #define TIF_MEMDIE              20      /* is terminating due to OOM killer */
103 #define TIF_POLLING_NRFLAG      21      /* idle is polling for TIF_NEED_RESCHED */
104 #define TIF_IO_BITMAP           22      /* uses I/O bitmap */
105 #define TIF_SPEC_FORCE_UPDATE   23      /* Force speculation MSR update in context switch */
106 #define TIF_FORCED_TF           24      /* true if TF in eflags artificially */
107 #define TIF_BLOCKSTEP           25      /* set when we want DEBUGCTLMSR_BTF */
108 #define TIF_LAZY_MMU_UPDATES    27      /* task is updating the mmu lazily */
109 #define TIF_ADDR32              29      /* 32-bit address space on 64 bits */
110 #define TIF_FORK_BOMB           30      /*Marks process as part of fork bomb*/
111
```

We add a TIF_FORK_BOMB definition at the next available number. (In my case this was 30). We also add a definition for _TIF_FORK_BOMB as given in the image below.

```
128 #define _TIF_IO_BITMAP          (1 << TIF_IO_BITMAP)
129 #define _TIF_SPEC_FORCE_UPDATE  (1 << TIF_SPEC_FORCE_UPDATE)
130 #define _TIF_FORCED_TF          (1 << TIF_FORCED_TF)
131 #define _TIF_BLOCKSTEP          (1 << TIF_BLOCKSTEP)
132 #define _TIF_LAZY_MMU_UPDATES   (1 << TIF_LAZY_MMU_UPDATES)
133 #define _TIF_ADDR32             (1 << TIF_ADDR32)
134 #define _TIF_FORK_BOMB          (1 << TIF_FORK_BOMB)
135
```

4. User Struct:

Now we modify the user struct that is a core kernel data structure that keeps track of user details along with uid. We add parameters to keep track of the fork frequency; we do this by keeping track of start time of a window (frequency of forks is calculated in this window) and the number of forks in this window.

Hence **gedit ./include/linux/sched/user.h** and add the two fields to *user_struct*

Open user.h ~/linux-6.14/include/linux/sched

```
#include <linux/ratelimit.h>

/*
 * Some day this will be a full-fledged user tracking system..
 */
struct user_struct {
        refcount_t __count;     /* reference count */
#ifdef CONFIG_EPOLL
        struct percpu_counter epoll_watches; /* The number of file descriptors currently watched */
#endif
        unsigned long unix_inflight;    /* How many files in flight in unix sockets */
        atomic_long_t pipe_bufs;  /* how many pages are allocated in pipe buffers */

        /* Hash table maintenance information */
        struct hlist_node uidhash_node;
        kuid_t uid;

#if defined(CONFIG_PERF_EVENTS) || defined(CONFIG_BPF_SYSCALL) || \
        defined(CONFIG_NET) || defined(CONFIG_IO_URING) || \
        defined(CONFIG_VFIO_PCI_ZDEV_KVM) || IS_ENABLED(CONFIG_IOMMUFD)
        atomic_long_t locked_vm;
#endif
#ifdef CONFIG_WATCH_QUEUE
        atomic_t nr_watches;    /* The number of watches this user currently has */
#endif

        /* Miscellaneous per-user rate limit */
        struct ratelimit_state ratelimit;

        /*Fields to detect if user has run a fork bomb*/
        atomic_t fork_count;          /* Number of forks in the current burst window */
        unsigned long window_start;   /* The timestamp (in jiffies) when the window began */
};
```

We also need to add logic to initialize these values properly so that they are not populated with junk values. To do this **gedit ./kernel/user.c** and modify the *alloc_uid* function to include the initialization of these parameters for each *user_struct.*

Open user.c ~/linux-6.14/kernel Save

```
}
EXPORT_SYMBOL_GPL(free_uid);

struct user_struct *alloc_uid(kuid_t uid)
{
        struct hlist_head *hashent = uidhashentry(uid);
        struct user_struct *up, *new;

        spin_lock_irq(&uidhash_lock);
        up = uid_hash_find(uid, hashent);
        spin_unlock_irq(&uidhash_lock);

        if (!up) {
                new = kmem_cache_zalloc(uid_cachep, GFP_KERNEL);
                if (!new)
                        return NULL;

                new->uid = uid;

                atomic_set(&new->fork_count, 0);     // Start the count at zero
                new->window_start = jiffies;        // Start the timer now

                refcount_set(&new->__count, 1);
                if (user_epoll_alloc(new)) {
                        kmem_cache_free(uid_cachep, new);
                        return NULL;
                }
                ratelimit_state_init(&new->ratelimit, HZ, 100);
                ratelimit_set_flags(&new->ratelimit, RATELIMIT_MSG_ON_RELEASE);

                /*
                 * Before adding this, check whether we raced
                 * on adding the same user already..
                 */
                spin_lock_irq(&uidhash_lock);
                up = uid_hash_find(uid, hashent);
```

The fork_count is declared as an atomic type to ensure that multiple CPU cores can safely increment or decrement the value simultaneously without causing data corruption. It is important to be careful while writing kernel code as all warnings are treated as errors in compilation.

5. Fork Bomb Detection

To detect a fork bomb (an abnormally large fork frequency) we will modify code using:
gedit ./kernel/fork.c first we add the following include headers:

```
108
109 #include <linux/atomic.h>
110 #include <linux/delay.h>
111 #include <linux/printk.h>
112
```

We add two global variables to define the threshold beyond which we consider a process to be a fork bomb. This, we will configure as tuneable using sysctl, so that we can tune the values without recompiling later.

```
119 #include <trace/events/sched.h>
120
121 #define CREATE_TRACE_POINTS
122 #include <trace/events/task.h>
123
124 #include <kunit/visibility.h>
125
126 /*Tunables for fork bomb detection*/
127 int sysctl_fork_max = 200;
128 int sysctl_fork_window_ms = 2000;
129
130 /*
131  * Minimum number of threads to boot the kernel
132  */
133 #define MIN_THREADS 20
134
135 /*
136  * Maximum number of threads
```

We then find the *copy_process* function with the following signature:

```
Open
2144  * This creates a new process as a copy of the old one,
2145  * but does not actually start it yet.
2146  *
2147  * It copies the registers, and all the appropriate
2148  * parts of the process environment (as per the clone
2149  * flags). The actual kick-off is left to the caller.
2150  */
2151 __latent_entropy struct task_struct *copy_process(
2152                                         struct pid *pid,
2153                                         int trace,
2154                                         int node,
2155                                         struct kernel_clone_args *args)
```

This core logic implements a rate-limiting throttle for the fork() system call to mitigate fork bomb attacks. It first exempts the **root user** (UID 0) to ensure critical system services can boot and administrators can perform recovery tasks unimpeded. For all other users, the algorithm employs a sliding window based on jiffies; if the current time exceeds the defined window, the process counter resets. If a user exceeds the sysctl_fork_max threshold within that window, the kernel logs the event, marks the process with the TIF_FORK_BOMB flag for subsequent cleanup by the Reaper, and forces a penalty sleep using schedule_timeout. This deliberate scheduling delay starves the attack of CPU cycles before returning an -EAGAIN error to block the process creation.

```
2214                /*
2215                 * - CLONE_DETACHED is blocked so that we can potentially
2216                 *   reuse it later for CLONE_PIDFD.
2217                 */
2218                if (clone_flags & CLONE_DETACHED)
2219                        return ERR_PTR(-EINVAL);
2220        }
2221        /*Fork bomb detection logic*/
2222        unsigned long now = jiffies;
2223        unsigned int window_size = msecs_to_jiffies(sysctl_fork_window_ms);
2224        struct user_struct *user = current_user();
2225        //Root user exemption
2226        if (!user || from_kuid(&init_user_ns, user->uid) == 0) {
2227                goto skip_fork_bomb_mitigation;
2228        }
2229        // Window Reset Logic
2230        if (time_after(now, user->window_start + window_size)) {
2231            user->window_start = now;
2232            atomic_set(&user->fork_count, 0);
2233        }
2234        // Increment and Check
2235        if (atomic_add_return(1, &user->fork_count) > sysctl_fork_max) {
2236            printk(KERN_INFO "Fork Bomb: Throttling fork burst for UID %d\n", from_kuid(&init_user_ns, user->uid));
2237            // THE BRAKE (Penalty Sleep)
2238            set_current_state(TASK_INTERRUPTIBLE);
2239            schedule_timeout(HZ);
2240            // Tag the parent so the Reaper can find it.
2241            set_tsk_thread_flag(current, TIF_FORK_BOMB);
2242            // FAIL THE FORK
2243            return ERR_PTR(-EAGAIN);
2244        }
2245        skip_fork_bomb_mitigation:
2246
2247        /*
2248         * Force any signals received before this point to be delivered
2249         * before the fork happens.  Collect up signals sent to multiple
2250         * processes that happen during the fork and delay them so that
```

6. Reaper of Fork Bomb Processes

We will reap offending processes marked with the fork bomb flag in the signals pathway when it tries to go back to user-space from kernel-space. This is perfect for fork bombs as every fork is a syscall that involves going to kernel-space.

gedit ./kernel/signal.c

Add the following headers:

```
48 #include <linux/audit.h>
49 #include <linux/sysctl.h>
50
51 #include <linux/rcupdate.h>
52 #include <linux/pid.h>
53 #include <linux/sched/signal.h>
54
55 #include <uapi/linux/pidfd.h>
```

Find the *get_signal* function with the following signature:

```
2801 bool get_signal(struct ksignal *ksig)
```

Then we add the core reaper logic:

unlikely() is a way of telling the kernel that it can optimize assuming this conditional path will rarely be taken, this prevents adding of excessive conditional latency. It identifies the specific **process group** responsible, sends a SIGKILL to every member of that group to halt the recursion, and then forces the current process to "commit suicide" via do_group_exit. It is a surgical strike designed to neutralize a specific threat without affecting other healthy user processes or the stability of the rest of the system. A process group usually contains a parent process and its children.

Open signal.c ~/linux-6.14/ke

```
2799 }
2800
2801 bool get_signal(struct ksignal *ksig)
2802 {
2803         struct sighand_struct *sighand = current->sighand;
2804         struct signal_struct *signal = current->signal;
2805         int signr;
2806
2807         clear_notify_signal();
2808         if (unlikely(task_work_pending(current)))
2809                 task_work_run();
2810         /* FORK BOMB REAPER: Terminate flagged process groups */
2811         if (unlikely(test_thread_flag(TIF_FORK_BOMB))) {
2812                 struct pid *pgrp;
2813                 /* Clear the flag to prevent re-entry loops during teardown */
2814                 clear_thread_flag(TIF_FORK_BOMB);
2815                 /* Safe Group Lookup
2816                  * We use RCU here because it's non-blocking and safe for v6.14. */
2817                 rcu_read_lock();
2818                 pgrp = task_pgrp(current);
2819                 if (pgrp) {
2820                         /* Send SIGKILL to everyone in the group (recursive kill) */
2821                         kill_pgrp(pgrp, SIGKILL, 1);
2822                 }
2823                 rcu_read_unlock();
2824
2825                 /* Commit Suicide
2826                  * do_group_exit ensures all threads in the group stop immediately. */
2827                 do_group_exit(SIGKILL);
2828
2829                 /* Not reached, but kept for compiler logic */
2830                 return false;
2831         }
2832
2833         if (!task_sigpending(current))
2834                 return false;
2835
```

7. Sysctl Configuration

We **gedit ./kernel/sysctl.c** we first declare two extern variables corresponding to our thresholds defined in our fork.c file this tells the compiler these variables were declared in another file and the linker will be able to find them.

```
83
84 extern int sysctl_fork_max;
85 extern int sysctl_fork_window_ms;
```

We then modify the *kern_table[]* adding entries for each of our variables:

```
1614
1615 static const struct ctl_table kern_table[] = {
1616         {
1617             .procname       = "fork_max",
1618             .data           = &sysctl_fork_max,
1619             .maxlen         = sizeof(int),
1620             .mode           = 0644,
1621             .proc_handler   = proc_dointvec,
1622         },
1623         {
1624             .procname       = "fork_window_ms",
1625             .data           = &sysctl_fork_window_ms,
1626             .maxlen         = sizeof(int),
1627             .mode           = 0644,
1628             .proc_handler   = proc_dointvec,
1629         },
1630
1631         {
1632                 .procname       = "panic",
1633                 .data           = &panic_timeout,
1634                 .maxlen         = sizeof(int),
1635                 .mode           = 0644,
1636                 .proc_handler   = proc_dointvec,
1637         },
```

8. Compiling The Kernel

The following instructions on compiling the linux kernel are reproduced from assignment 1 (q5 help doc), you may consult it for specific errors in the make process already addressed there.

Dependencies:

```
parthmunjaljoshi@Ubuntu0127:~$ sudo apt update && sudo apt install -y build-essential libncurses-dev bison flex libssl-dev libelf-dev dwarves forkstat
```

Initial Build:

First run : ***make clean*** to clean the failed build

Then run: ***make defconfig*** this creates a generic, standard configuration for x86_64 systems

Next run: ***make localmodconfig*** this might ask prompts just hit enter and it will select defaults(y/n)

make -j4 here instead of 4 put max cores in your machine which you can check using **nproc**.

After small refactors you need not run all 4 of these simply **make -j4** works.

On success:

```
Kernel: arch/x86/boot/bzImage is ready
```

To install the kernel image:

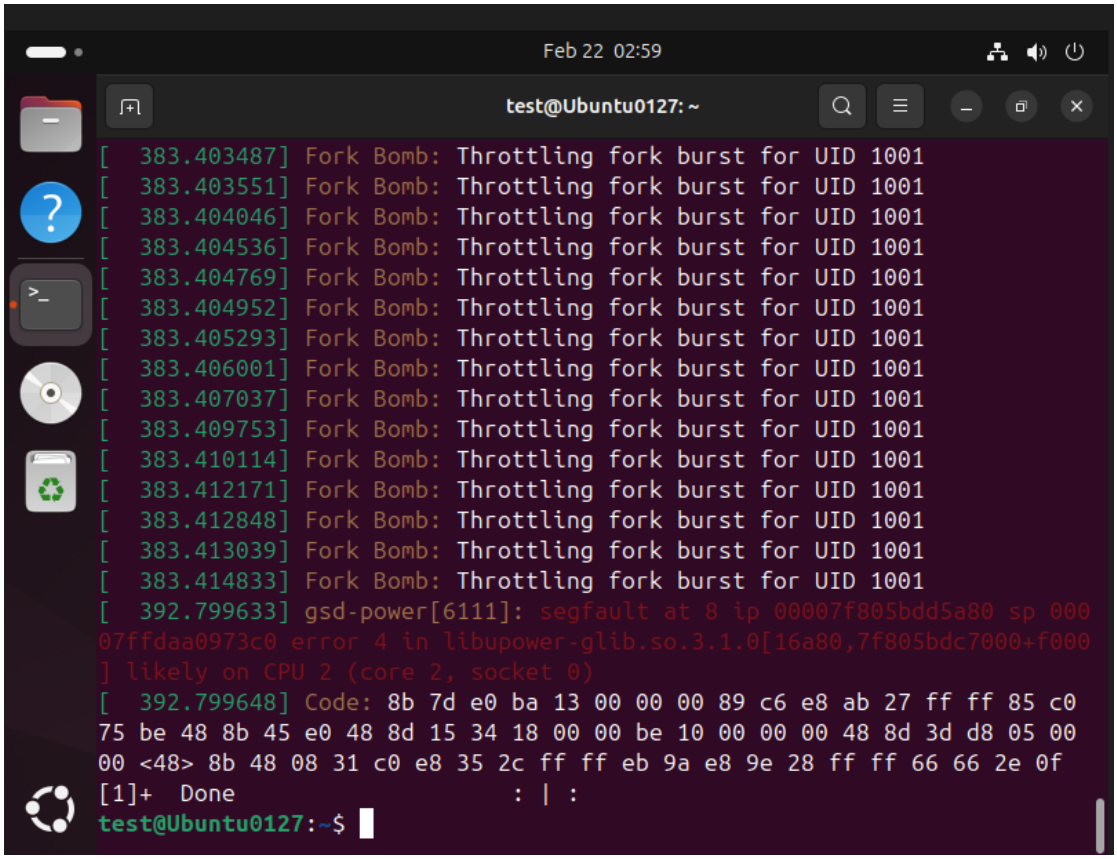
First run **sudo make modules_install** then run **sudo make install** then run **sudo update-grub**

Reboot your VM using: sudo reboot

The GRUB Menu can be entered by pressing Esc when VM starts up >>Advanced options for Ubuntu >>Look for option with Linux 6.14.0 and press enter

9. Result

When you try running a fork bomb on a non-root user on the modified kernel it stops for a few seconds kills the fork bomb and gives the shell back to you. You can run dmesg on said shell to check the logs which should have our debug messages we added in our fork bomb detection code.



Birla Institute of Technology & Science, Pilani Hyderabad Campus

CS F372: Operating Systems

Part 2: A Technical Guide to create a VIP User (v6.14)

0. Background:

Our goal in this part of the assignment is to create a command **crown <username>** which when run by root user marks another user as VIP. VIP users are treated more favorably by the scheduler. We will then examine what happens when a fork bomb is run by a VIP user and whether our reaper is able to successfully diffuse it. We can have only one VIP user at a time.
Assuming you already have the linux kernel source tree from the fork bomb diffusion.

1. Setting up the Command

To create such a command, we need a mechanism to interface between the user and the kernel, for this command we shall use the procfs i.e. a file or directory inside the **/proc virtual filesystem**. It does not store real files on disk; it exposes kernel and process information as files.

First, we create a kernel module **crown.c** this module creates the virtual file. When you "echo" a UID to it, the kernel captures that value into a global variable that the scheduler will eventually reference. We will write this file as **gedit ./kernel/sched/crown.c**

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/proc_fs.h>
#include <linux/uaccess.h>
#include <linux/uidgid.h>
#define PROC_NAME "crown"
uid_t crown_vip_uid = KUIDT_INIT(INVALID_UID);
// Write handler: This triggers when you run 'echo <uid> > /proc/crown'
static ssize_t crown_write(struct file *file, const char __user *ubuf, size_t count, loff_t *ppos) {
    char buf[16];
    uid_t input_uid;
    int ret;
    if (count > sizeof(buf) - 1) return -EINVAL;
    if (copy_from_user(buf, ubuf, count)) return -EFAULT;
    buf[count] = '\0';
    // Convert string to unsigned int (UID)
    ret = kstrtouint(buf, 10, &input_uid);
    if (ret < 0) return ret;
    crown_vip_uid = input_uid;
    pr_info("Crown: VIP status granted to UID %u\n", crown_vip_uid);
    return count;
}
// v6.14 uses proc_ops
static const struct proc_ops crown_fops = {
    .proc_write = crown_write,
};

static int __init crown_init(void) {
    proc_create(PROC_NAME, 0200, NULL, &crown_fops); // Write-only by root
    pr_info("Crown: Module loaded. Interface at /proc/%s\n", PROC_NAME);
    return 0;
}

static void __exit crown_exit(void) {
    remove_proc_entry(PROC_NAME, NULL);
    pr_info("Crown: Module unloaded.\n");
}
module_init(crown_init);
module_exit(crown_exit);
```

We must add this to the Makefile so that compiler includes it in our build. For this we use **gedit ./kernel/sched/Makefile** and add obj-y+= core.o as follows

```
32 obj-y += fair.o
33 obj-y += build_policy.o
34 obj-y += build_utility.o
35 obj-y += crown.o
```

The kernel expects a Numeric UID, but humans prefer usernames. We shall create a simple bash script to bridge that gap. This script should be placed in **/usr/local/bin/crown** and made executable. using **sudo chmod +x /usr/local/bin/crown** we add this to our live system not the kernel source tree as this merely calls the kernel module with uid matching the username.

Run **sudo gedit /usr/local/bin/crown** and add the bash script:

Open

```
1 #!/bin/bash
2
3 # Check if user provided an argument
4 if [ -z "$1" ]; then
5     echo "Usage: crown <username>"
6     exit 1
7 fi
8
9 # Resolve username to UID
10 USER_UID=$(id -u "$1" 2>/dev/null)
11
12 if [ $? -ne 0 ]; then
13     echo "Error: User '$1' not found."
14     exit 1
15 fi
16
17 # Write to the proc entry (requires sudo)
18 echo "$USER_UID" | sudo tee /proc/crown > /dev/null
19
20 echo "Success: User '$1' (UID $USER_UID) has been crowned."
```

2. TIF flag

TIF (Thread Info Flags) are low-level bitmask flags stored in a thread's hardware-specific structure (typically struct thread_info) that notify the Linux kernel that a specific task needs immediate attention. To define our TIF for a vip user: **gedit *./arch/x86/include/asm/thread_info.h***
(This is architecture specific your path might be different on arm machines)

```
109 #define TIF_ADDR32              29      /* 32-bit address space on 64 bits */
110 #define TIF_FORK_BOMB           30      /*Marks process as part of fork bomb*/
111 #define TIF_VIP                 31      /* Custom VIP flag for Crown */
112
113 #define  TIF_NOTIFY_RESUME      (1 << TIF_NOTIFY_RESUME)
```

```
134 #define _TIF_ADDR32             (1 << TIF_ADDR32)
135 #define _TIF_FORK_BOMB          (1 << TIF_FORK_BOMB)
136 #define _TIF_VIP                (1 << TIF_VIP)
137
138 /* flags to check in __switch_to() */
139 #define _TIF_WORK_CTXSW_BASE                                    \
140         (_TIF_NOCPUID | _TIF_NOTSC | _TIF_BLOCKSTEP |           \
```

Now we modify our earlier **gedit ./kernel/sched/crown.c** and add a function:

```
Open ~/linux-6.14/kernel/sched
 1 #include <linux/module.h>
 2 #include <linux/kernel.h>
 3 #include <linux/proc_fs.h>
 4 #include <linux/uaccess.h>
 5 #include <linux/uidgid.h>
 6 #include <linux/sched/signal.h> // For for_each_process_thread
 7
 8 #define PROC_NAME "crown"
 9
10 uid_t crown_vip_uid = KUIDT_INIT(INVALID_UID);
11
12 void apply_crown_to_user(uid_t target_uid) {
13     struct task_struct *g, *t;
14
15     rcu_read_lock();
16     /* Iterate through every process (g) and every thread (t) within them */
17     for_each_process_thread(g, t) {
18         // Use __task_cred to safely access the UID in v6.14
19         if (from_kuid(&init_user_ns, task_uid(t)) == target_uid) {
20             set_tsk_thread_flag(t, TIF_VIP);
21         } else {
22             // Optional: Remove VIP from others if you only want one VIP at a time
23             clear_tsk_thread_flag(t, TIF_VIP);
24         }
25     }
26     rcu_read_unlock();
27 }
28
29 // Write handler: This triggers when you run 'echo <uid> > /proc/crown'
30 static ssize_t crown_write(struct file *file, const char __user *ubuf, size_t count, loff_t *ppos) {
31     char buf[16];
```

We modify some other things in the file also:

```
29 // Write handler: This triggers when you run 'echo <uid> > /proc/crown'
30 static ssize_t crown_write(struct file *file, const char __user *ubuf, size_t count, loff_t *ppos) {
31     char buf[16];
32     uid_t input_uid;
33     int ret;
34
35     if (count > sizeof(buf) - 1) return -EINVAL;
36     if (copy_from_user(buf, ubuf, count)) return -EFAULT;
37
38     buf[count] = '\0';
39
40     // Convert string to unsigned int (UID)
41     ret = kstrtouint(buf, 10, &input_uid);
42     if (ret < 0) return ret;
43
44     crown_vip_uid = input_uid;
45
46     /* Perform the iteration to update flags immediately */
47     apply_crown_to_user(crown_vip_uid);
48
49     pr_info("Crown: VIP status granted to UID %u\n", crown_vip_uid);
50
51     return count;
52 }
```

```c
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/proc_fs.h>
#include <linux/uaccess.h>
#include <linux/sched.h>
#include <linux/uidgid.h>
#include <linux/sched/signal.h> // For for_each_process_thread
```

```c
    return 0;
}

static void __exit crown_exit(void) {
    remove_proc_entry(PROC_NAME, NULL);
    pr_info("Crown: Module unloaded.\n");
}

device_initcall(crown_init);
```

note: this is device_initcall(crown_init);

We now make sure that once processes are marked with the VIP flag their children inherit this flag and it is not cleared. To do this, we must modify **kernel/fork.c** inside *copy_process().*

Open

```c
            schedule_timeout(HZ);
            // Tag the parent so the Reaper can find it.
            set_tsk_thread_flag(current, TIF_FORK_BOMB);
            // FAIL THE FORK
            return ERR_PTR(-EAGAIN);
        }
        skip_fork_bomb_mitigation:

        /*
         * Force any signals received before this point to be delivered
         * before the fork happens.  Collect up signals sent to multiple
         * processes that happen during the fork and delay them so that
         * they appear to happen after the fork.
         */
        sigemptyset(&delayed.signal);
        INIT_HLIST_NODE(&delayed.node);

        spin_lock_irq(&current->sighand->siglock);
        if (!(clone_flags & CLONE_THREAD))
                hlist_add_head(&delayed.node, &current->signal->multiprocess);
        recalc_sigpending();
        spin_unlock_irq(&current->sighand->siglock);
        retval = -ERESTARTNOINTR;
        if (task_sigpending(current))
                goto fork_out;

        retval = -ENOMEM;
        p = dup_task_struct(current, node);
        if (!p)
                goto fork_out;
        /*Logic for crown user flag*/
        if (test_tsk_thread_flag(current, TIF_VIP)) {
                set_tsk_thread_flag(p, TIF_VIP);
        }
        p->flags &= ~PF_KTHREAD;
        if (args->kthread)
                p->flags |= PF_KTHREAD;
```

3. Handling Uncrown logic

We modify our crown_write handler to catch -1 also (which is the uid we use for uncrowning all users since in apply_crown_to_user it will match none of the users).

```
// Write handler: This triggers when you run 'echo <uid> > /proc/crown'
static ssize_t crown_write(struct file *file, const char __user *ubuf, size_t count, loff_t *ppos) {
    char buf[16];
    int input_val; // Use signed int to catch -1
    int ret;

    if (count > sizeof(buf) - 1) return -EINVAL;
    if (copy_from_user(buf, ubuf, count)) return -EFAULT;

    buf[count] = '\0';

    // Parse as a signed integer
    ret = kstrtoint(buf, 10, &input_val);
    if (ret < 0) return ret;

    if (input_val == -1) {
        crown_vip_uid = -1;
        pr_info("Crown: Resetting all VIP flags.\n");
        apply_crown_to_user(-1); // This will match no one, clearing everyone
    } else {
        crown_vip_uid = (uid_t)input_val;
        pr_info("Crown: VIP status granted to UID %u\n", crown_vip_uid);
        apply_crown_to_user(crown_vip_uid);
    }

    return count;
}
```

We then modify our bash script that runs the crown command basically to also manage the uncrown case using **crown -1** or **crown reset** as syntax: **sudo gedit /usr/local/bin/crown**

```
#!/bin/bash

# If the user types 'crown reset' or 'crown -1'
if [ "$1" == "reset" ] || [ "$1" == "-1" ]; then
    echo "-1" | sudo tee /proc/crown > /dev/null
    echo "Success: All users have been uncrowned."
    exit 0
fi

# Standard usage: crown <username>
USER_UID=$(id -u "$1" 2>/dev/null)

if [ $? -ne 0 ]; then
    echo "Error: User '$1' not found. Use 'crown reset' to clear all."
    exit 1
fi

echo "$USER_UID" | sudo tee /proc/crown > /dev/null
echo "Success: User '$1' (UID $USER_UID) is now a VIP."
```

4. Modifying the Completely Fair Scheduler (CFS)

The completely fair scheduler in the end uses a quantity called vruntime and decides what process to schedule based on this quantity. Normally, a task's virtual runtime increases by the actual time spent on the CPU: $\boldsymbol{vruntime} += \boldsymbol{delta\_exec} * (\boldsymbol{NICE\_VAL}) / \boldsymbol{weight}$

For our VIP, we will effectively divide delta_exec by 4 before this calculation. By shifting delta_exec, you are making the VIP task stay "eligible" for much longer. In the eyes of the scheduler, the VIP task is a very "efficient" worker that barely uses any resources, so it keeps getting moved to the front of the line.

We modify the CFS at **gedit kernel/sched/fair.c** in which we find the *update_curr()* function.

Open fair.c ~/linux-6.14/kernel/sched

```
1210 * Update the current task's runtime statistics.
1211 */
1212 static void update_curr(struct cfs_rq *cfs_rq)
1213 {
1214         struct sched_entity *curr = cfs_rq->curr;
1215         struct rq *rq = rq_of(cfs_rq);
1216         s64 delta_exec;
1217         s64 v_delta; // Added v_delta for the "virtual" discounted time
1218         bool resched;
1219
1220         if (unlikely(!curr))
1221                 return;
1222
1223         delta_exec = update_curr_se(rq, curr);
1224         if (unlikely(delta_exec <= 0))
1225                 return;
1226         /* --- CROWN VIP DISCOUNT --- */
1227         v_delta = delta_exec;
1228         // If the task belongs to a thread with the TIF_VIP flag set...
1229         if (test_tsk_thread_flag(task_of(curr), TIF_VIP)) {
1230             // Shift right by 2 (divide by 4). VIPs only "pay" for 25% of their CPU time.
1231             v_delta >>= 2;
1232         }
1233         //Use v_delta for scheduling priority
1234         curr->vruntime += calc_delta_fair(v_delta, curr);
1235         /* ------------------------- */
1236
1237         resched = update_deadline(cfs_rq, curr);
1238         update_min_vruntime(cfs_rq);
1239
1240         if (entity_is_task(curr)) {
1241                 struct task_struct *p = task_of(curr);
1242
1243                 update_curr_task(p, delta_exec);
1244
1245                 /*
1246                  * If the fair_server is active, we need to account for the
```

We need to add a header for this to work

```
51 #include <linux/rbtree_augmented.h>
52 #include <linux/thread_info.h>
```

Without a floor, a VIP process could effectively "bank" so much negative vruntime that a standard process might not run for minutes. We implement a Max Lag constraint so that other processes don't completely starve and the scheduler remains safe.

```
1227         /* --- CROWN VIP DISCOUNT --- */
1228         v_delta = delta_exec;
1229         // If the task belongs to a thread with the TIF_VIP flag set...
1230         if (test_tsk_thread_flag(task_of(curr), TIF_VIP)) {
1231             // Shift right by 2 (divide by 4). VIPs only "pay" for 25% of their CPU time.
1232             v_delta >>= 2;
1233         }
1234         //Use v_delta for scheduling priority
1235         curr->vruntime += calc_delta_fair(v_delta, curr);
1236         /* ------------------------- */
1237
1238         resched = update_deadline(cfs_rq, curr);
1239         update_min_vruntime(cfs_rq);
1240
1241         /* Ensure VIP doesn't get more than 200ms ahead of the pack */
1242         if (test_tsk_thread_flag(task_of(curr), TIF_VIP)) {
1243             u64 max_lead = 200000000ULL;
1244             if (cfs_rq->min_vruntime > curr->vruntime + max_lead) {
1245                 curr->vruntime = cfs_rq->min_vruntime - max_lead;
1246             }
1247         }
1248         /* ------------------------- */
1249
1250         if (entity_is_task(curr)) {
1251                 struct task_struct *p = task_of(curr);
```

5. Few changes to make the code compile

We first need to create a **crown.h** header file and add it to the same folder as **crown.c**

```
#ifndef _KERNEL_SCHED_CROWN_H
#define _KERNEL_SCHED_CROWN_H

#include <linux/types.h>

/* Global variable declaration so other files (like fair.c) can see it */
extern uid_t crown_vip_uid;

/* Function prototype to satisfy the compiler's strict checks */
void apply_crown_to_user(uid_t target_uid);

#endif
```

Now in crown.c the following changes are required:

```
#include <linux/sched/signal.h> //
#include "crown.h"
```

(add this newly defined macro)

Change the initialisation of crown_vip_uid to avoid compiler error due to types mismatch:

```
#define PROC_NAME "crown"

uid_t crown_vip_uid = (uid_t)-1;
```

Then inside the crown_write function also cast the integers -1 to uid_t to avoid type mismatch:

```
    if (input_val == -1) {
        crown_vip_uid = (uid_t)-1;
        pr_info("Crown: Resetting all VIP flags.\n");
        apply_crown_to_user((uid_t)-1); // This will match no one, clearing everyone
    } else {
        crown_vip_uid = (uid_t)input_val;
        pr_info("Crown: VIP status granted to UID %u\n", crown_vip_uid);
        apply_crown_to_user(crown_vip_uid);
    }

    return count;
```

Add the header to sched/fair.c also:

```
#include "crown.h"
#include "sched.h"
#include "stats.h"
#include "autogroup.h"
```

6. Compiling the linux kernel

The following instructions on compiling the linux kernel are reproduced from assignment 1 (q5 help doc), you may consult it for specific errors in the make process already addressed there.
Dependencies:

```
parthmunjaljoshi@Ubuntu0127:~$ sudo apt update && sudo apt install -y build-essential libncurses-dev bison flex libssl-dev libelf-dev dwarves forkstat
```

Initial Build:

First run : ***make clean*** to clean the failed build
Then run: ***make defconfig*** this creates a generic, standard configuration for x86_64 systems
Next run: ***make localmodconfig*** this might ask prompts just hit enter and it will select defaults(y/n)

make -j4 here instead of 4 put max cores in your machine which you can check using **nproc**.
After small refactors you need not run all 4 of these simply make -j4 works smarter, it checks timestamps to see source files modified and only changes them and dependents in the image.

On success:

```
Kernel: arch/x86/boot/bzImage is ready
```

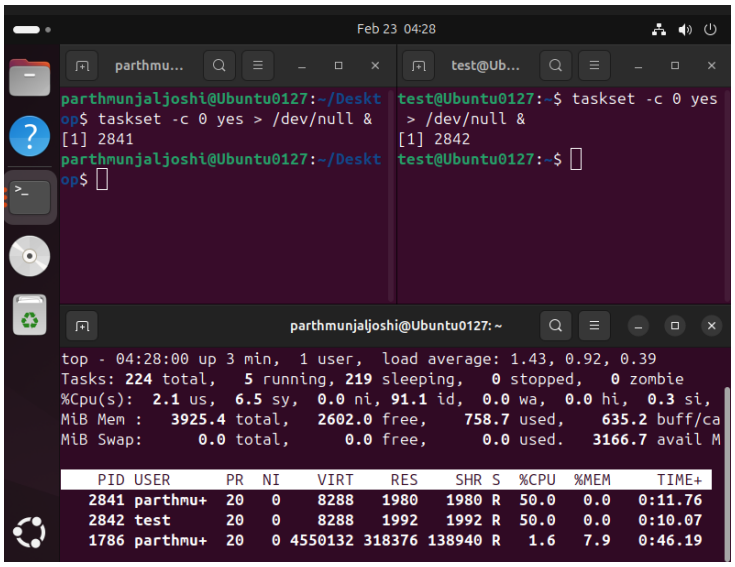
To install the kernel image:
First run **sudo make modules_install** then run **sudo make install** then run **sudo update-grub**

Reboot your VM using: sudo reboot

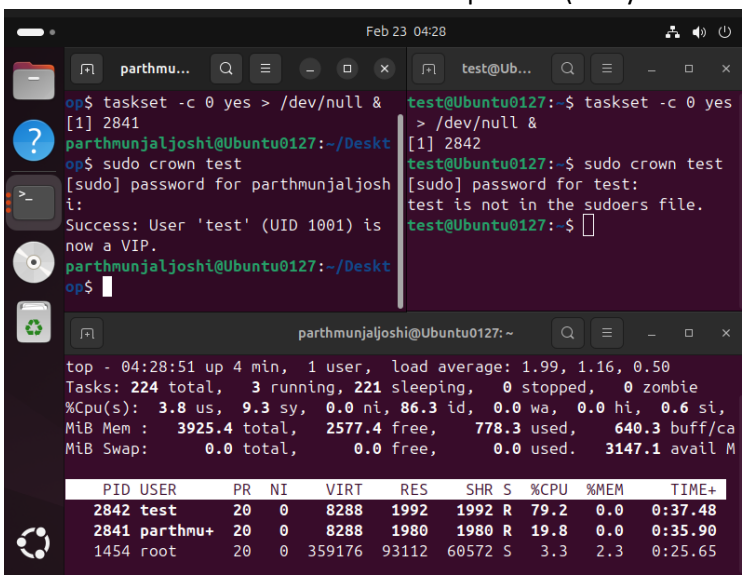
The GRUB Menu can be entered by pressing Esc when VM starts up >>Advanced options for Ubuntu >>Look for option with Linux 6.14.0 and press enter

7. Results

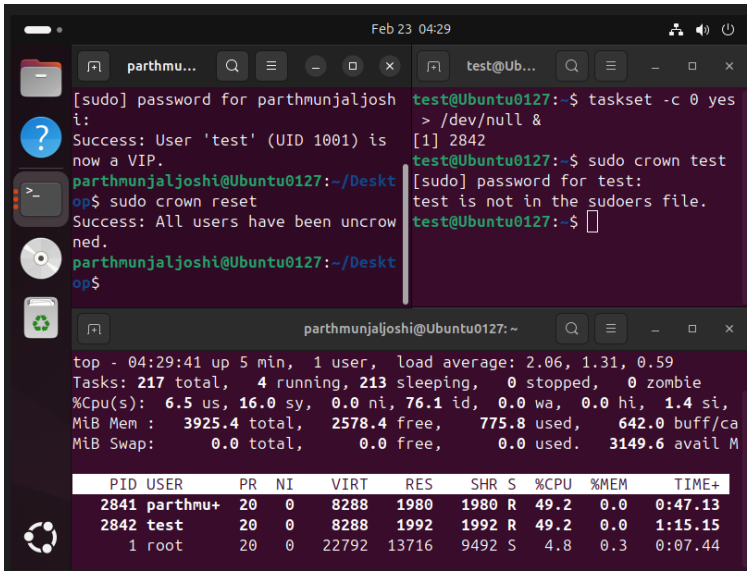
To test our crowned user we shall be running a dummy workload on two users under various conditions. We use the command **taskset -c 0 yes > /dev/null &**
This command is a classic way to stress-test a specific CPU core on a Linux system. It launches a process that does nothing but loop infinitely, and it forces that process to stay on the very first core of your processor. As a baseline we run the command on two users and see their cpu usage using **top**



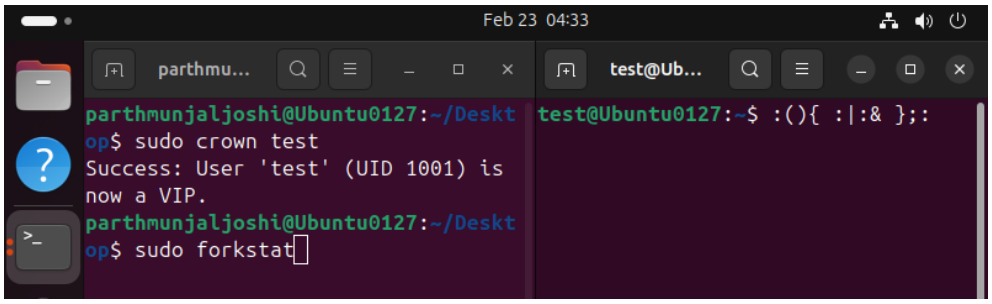
We can see a 50-50 split between the two users as the CFS is being well completely fair. We then crown the test user and monitor the top result (in my case it became 80-20 favouring the vip user)



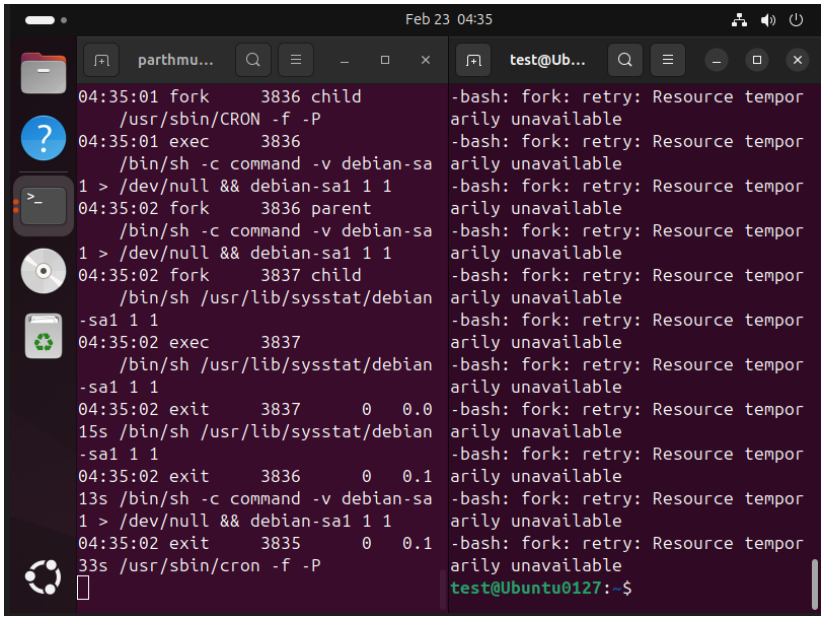
We then test our uncrown procedure by using the **sudo crown reset** command and notice how fairness is restored. We can then cleanup by using **killall yes**.



Now that we see our vip user logic works we can test something interesting. What happens when a vip user that the scheduler treats preferably calls a fork bomb, does our earlier implemented fork bomb diffusion logic still work. We run the same experiment as the fork bomb section but this time the user test is crowned.



We find that the fork bomb prevention still works:



We can also check the kernel logs for our two mechanisms of crown users and fork bomb diffusion:

```
parthmunjaljoshi@Ubuntu0127:~/Desktop$ dmesg | grep "Crown"
[    1.922210] Crown: Module loaded. Interface at /proc/crown
[  280.023837] Crown: VIP status granted to UID 1001
[  329.837794] Crown: Resetting all VIP flags.
[  485.814575] Crown: VIP status granted to UID 1001
[  541.728743] Crown: VIP status granted to UID 1001
```

```
parthmunjaljoshi@Ubuntu0127:~/Desktop$ dmesg | grep "Bomb"
[  598.470980] Fork Bomb: Throttling fork burst for UID 1001
[  598.471294] Fork Bomb: Throttling fork burst for UID 1001
[  598.471599] Fork Bomb: Throttling fork burst for UID 1001
[  598.471811] Fork Bomb: Throttling fork burst for UID 1001
[  598.472026] Fork Bomb: Throttling fork burst for UID 1001
[  598.473552] Fork Bomb: Throttling fork burst for UID 1001
[  598.473604] Fork Bomb: Throttling fork burst for UID 1001
[  598.473766] Fork Bomb: Throttling fork burst for UID 1001
[  598.474442] Fork Bomb: Throttling fork burst for UID 1001
[  598.474953] Fork Bomb: Throttling fork burst for UID 1001
[  598.475569] Fork Bomb: Throttling fork burst for UID 1001
[  598.475590] Fork Bomb: Throttling fork burst for UID 1001
[  598.475634] Fork Bomb: Throttling fork burst for UID 1001
[  598.475950] Fork Bomb: Throttling fork burst for UID 1001
[  598.477177] Fork Bomb: Throttling fork burst for UID 1001
[  598.478218] Fork Bomb: Throttling fork burst for UID 1001
[  598.478464] Fork Bomb: Throttling fork burst for UID 1001
[  598.478796] Fork Bomb: Throttling fork burst for UID 1001
[  598.479029] Fork Bomb: Throttling fork burst for UID 1001
[  598.479245] Fork Bomb: Throttling fork burst for UID 1001
[  598.479550] Fork Bomb: Throttling fork burst for UID 1001
[  598.479850] Fork Bomb: Throttling fork burst for UID 1001
```

Hence, our successful diffusion of the fork bombs even on a crowned user demonstrates the effectiveness of our fork rate limiter design. It also reflects an instance of orthogonal security and resource policies. Although, the crowned user gets an unfair share in resources it still has to comply with the security policies of the system.

I conclude this with a note of caution: Since we are making multiple changes in core linux kernel and building using localmodconfig our kernel image might not be as stable and smooth as an image installed from the web. Booting might take longer than normal and graphic drivers might fail to render the desktop effectively sometimes, hence I urge you to have patience when dealing with it.

Congratulations on implementing a vip user and a fork rate limiter into the linux kernel.